

AYUDA A LA PRÁCTICA 1. HTML Y CSS

— por si la necesitas en el desarrollo de la práctica —

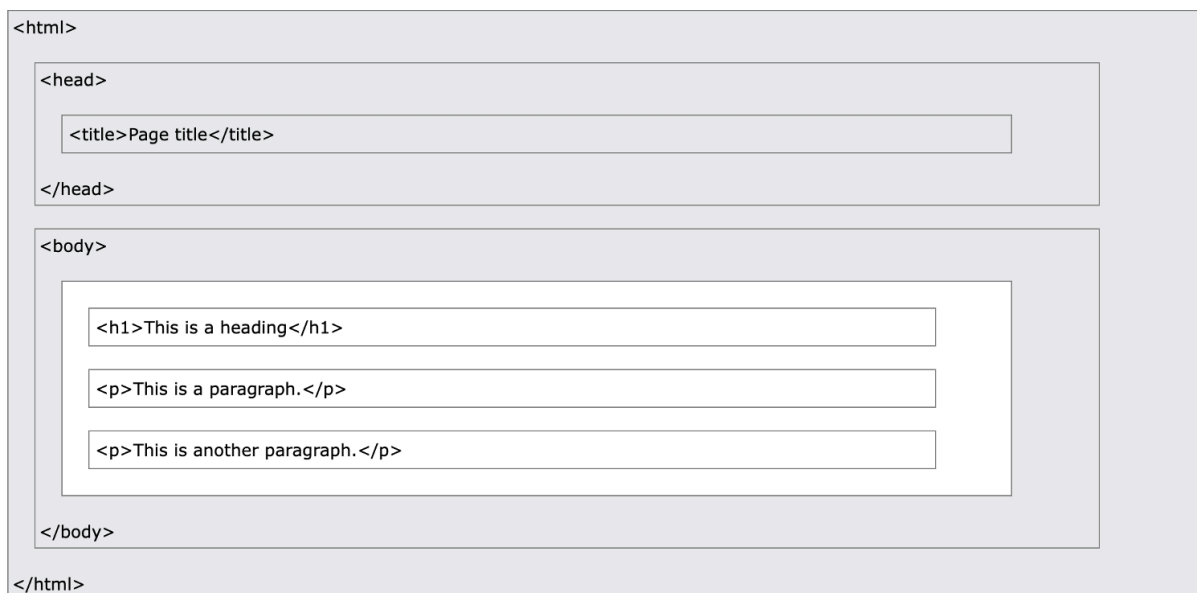
En clase de teoría veremos más elementos de HTML y CSS, pero para comenzar a realizar la práctica, haremos un repaso por lo más relevante para solucionar nuestro problema.

1. Esqueleto de una web (sin estilo)

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```



El contenido de la sección <body> se mostrará en un navegador. El contenido dentro del elemento <title> se mostrará en la barra de título del navegador o en la pestaña de la página.

2. Estructura de una página web (Layout).

Existen elementos semánticos, que definen las partes de una página web, que además indican a qué contenidos se refieren.

Ejemplo de layout con elementos con significado:



<header> - Define un encabezado para un documento o una sección.

<nav> - Define un conjunto de enlaces de navegación.

<section> - Define una sección en un documento

<article> - Define un contenido independiente y autónomo

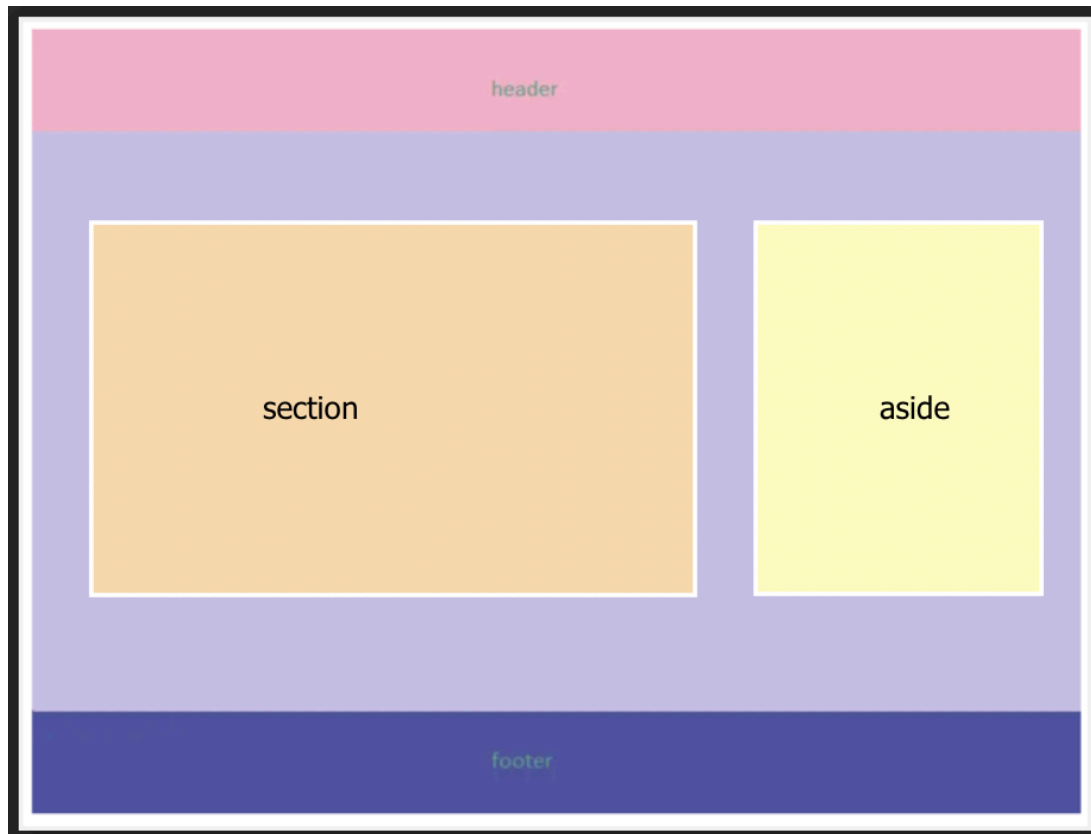
<aside> - Define contenido aparte del contenido (como una barra lateral)

<footer> - Define un pie de página para un documento o una sección

<details> - Define detalles adicionales que el usuario puede abrir y cerrar bajo demanda

<summary> - Define un encabezado para el elemento <details>.

Ejemplo de layout, en Html.



```
<!DOCTYPE html>
<html>
<body>
  <header>
    <meta charset="UTF-8">
    <h1>My Blog</h1>
  </header>
  <nav>
    <a href="#">Home</a> | <a href="#">About</a> | <a href="#">Contact</a>
  </nav>
  <section>
    <h2>Latest Posts</h2>
    <article>
      <h3>Post Title</h3>
      <p>This is a brief introduction to the blog post.</p>
    </article>
  </section>
  <aside>
    <h2>About Me</h2>
    <p>Short bio or profile information.</p>
  </aside>
  <footer>
    <p>&copy; 2024 My Blog</p>
  </footer>
</body>
</html>
```

Evidentemente el resultado no tiene ningún formato (el establecido por defecto por el navegador).

My Blog

[Home](#) | [About](#) | [Contact](#)

Latest Posts

Post Title

This is a brief introduction to the blog post.

About Me

Short bio or profile information.

© 2024 My Blog

Tendríamos que utilizar CSS para darle el formato deseado.

3. Idioma de una página web.

- Definir un lenguaje. Nuestra página web estará en español, completamente, no por secciones... por tanto.

```
<!DOCTYPE html>
<html lang="es"> ...
```

- ### 4. Cómo hacer una página responsive.
- Para ello hay que añadir esta etiqueta a todas las páginas web. Se incluye en el elemento <head>

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

- ### 5. Cómo poner títulos.
- Se incluyen en el head.

```
<head>
  <title>HTML Tutorial</title>
</head>
```

6. Elementos de bloques <div> o para organizar contenidos

Se pueden incluir contenidos con elementos dentro de una línea (por tanto, no habrá salto de línea incluido). Uno de los más usados es <div>

- `<span>` organiza un bloque de elementos, dentro de una misma línea. Ocupa el espacio que necesita, exclusivamente. Es muy usado, cuando quieres incluir muchas cosas de corrido.

Otros elementos se organizan cada uno en líneas diferentes, como o <p>.

- `<p>` define un párrafo
- `<div>` define una división o sección (es más usado).

7. Cómo crear una lista

```
<ul>
  <li>Opción 1</li>
  <li>Opción 2</li>
  <li>Opción 3</li>
</ul>
```

8. Cómo separar el estilo de la web

```
<head>
<link rel="stylesheet" href="/style.css">
</head>
```

9. Los CSS, elementos class e id

```
<div class="city" id="info_london">
  <h2>London</h2>
  <p>London is the capital of England.</p>
</div>
```

10. Comentarios en CSS y HTML

```
<!-- esto es un comentario de html -->
/* Lesto es un comentario de css*/
```

11. Selectores en CSS

- Para un elemento

```
p {
  text-align: center;
  color: red;
}
```

- Para una clase

```
.clase1 {
  text-align: center;
  color: red;
}
```

- Para un id

```
#id_ej {
  text-align: center;
  color: red;
}
```

- Universal


```
* {
  text-align: center;
  color: blue;
}
```
- Varios a la vez

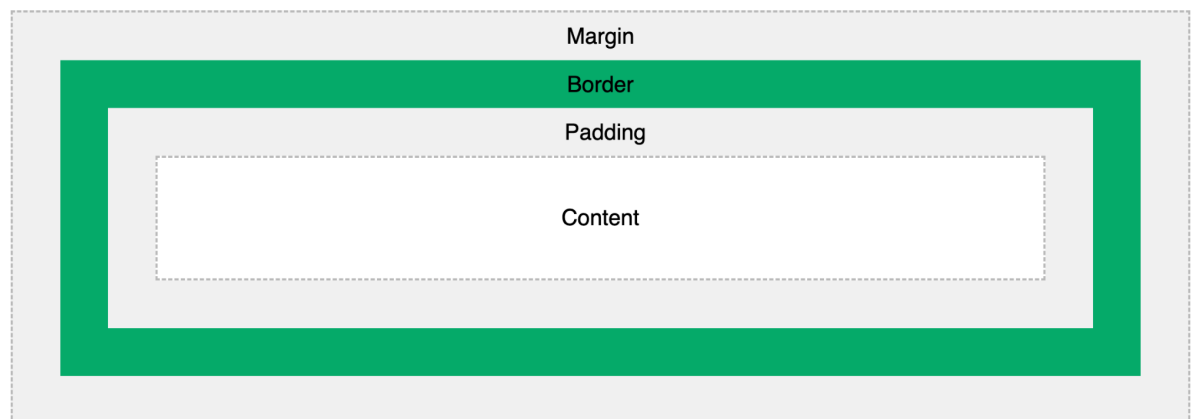

```
h1, h2, p {
  text-align: center;
  color: red;
}
```
- Hay otros avanzados... ver tutorial de clase de teoría!!.

12. Formato a un elemento CSS

```
<div class="city" id="info_london">
  <h2>London</h2>
  <p>London is the capital of England.</p>
</div>
```

```
.clase1 {
  text-align: center;
  color: red;
  font-family: "Lucida Console"
  background-color: lightgrey;
  color: blue;
  text-decoration: underline;
  font-style: italic;
  font-size: 14px;
}
```

13. Modelo de caja de CSS



```
div {
  width: 300px;
```

```
border: 15px solid green;
padding: 50px;
margin: 20px;
}
```

14. Cómo organizar los elementos en una página: display

El atributo **display** puede tener tres valores básicos: inline, block, inline-block

- Primera inline

```
span.a {
  display: inline; /* the default for span */
  width: 100px;
  height: 100px;
  padding: 5px;
  border: 1px solid blue;
  background-color: yellow;
}
```

En html

```
<div> Pruebas <span class="a"> Hola </span> <span class="a">
Hola </span> final de pruebas. </div>
```

Resultado:

Pruebas Hola Hola final de pruebas.

- Segunda block

```
display: block;
```

- Tercera inline-block

Pruebas

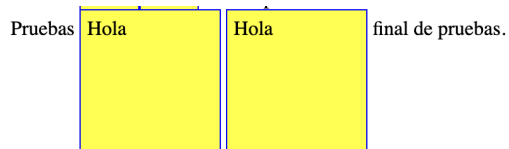
Hola

Hola

final de pruebas.

- Tercera inline-block

```
display: inline-block;
```



Sin embargo, esto no nos permite nada de flexibilidad.

15. Cómo organizar los elementos con Flex

display: flex o inline-flex;

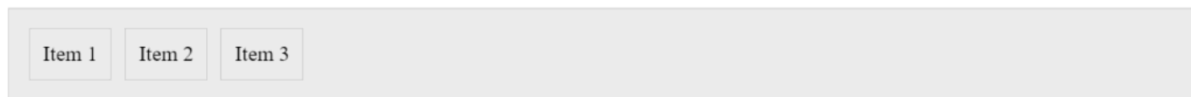
Los elementos se organizan bien en filas o en columnas exclusivamente.

Sin embargo, permite adaptarse al contenido de forma flexible.

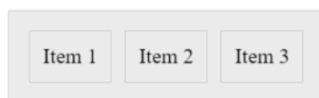
Se pueden anidar.

En la caja contenedora, se definen los atributos:

display: flex



display: inline-flex



Organización de los elementos:

- **flex-direction:** row / row-reverse / column /column-reverse (decide si se organizan los elementos en filas o en columnas, y en qué orden)
- **direction:** rtl /ltr (esto no lo vamos a cambiar)
- **flex-wrap:** wrap/nowrap/wrap-reverse (permiten moverse los elementos en lugar de achicarse).

flex-wrap: no-wrap;



flex-wrap: wrap;



- **justify-content:** start/end/center/space-between/space-around/**space-evenly** (alineación eje principal)
- **align-items:** stretch/start/end/center/**baseline** (alineación eje cruzado- solo una línea)

- align-content: start/space-around/space-between→ (cuando hay varias filas, organiza el contenido, en el eje cruzado)
- gap: medida (distancia los elementos entre sí, mejor que usar padding o margin)

<https://www.eniun.com/flexbox-modelo-caja-flexible-css/>

Con cada uno de los items, también se puede ajustar algo:

- order: 0→orden visualización
- flex-grow: → 0 /1 /2/ n .. (indicamos que SI permitimos que crezca, y en la proporción dada por n), necesita indicar flex-basis.
- flex-shrink → 0 /1 /2/ n .. (capacidad para encogerse)
- flex-basis auto/0/1/2/ n (ancho de cada elemento antes de encogerse especificado por el tamaño mínimo de pixels)
- align-self: stretch/top, end/baseline/.. (para coger un solo elemento, solo funciona con la propiedad align-items)
- o se puede poner conjunto: tres valores: flex: flex-grow | flex-shrink | flex-basis)
 - flex: 2 2 10%;

16. Ejemplo con Flex para crear una cabecera responsive

Ejemplo para crear un menú superior, utilizando una lista, cambiándole la orientación, e indicando

```
header { display: flex; flex-wrap: wrap; }
header .logo {
  flex: 0 0 300px; /* Set a fixed width for the logo */
}
header .menu {
  flex: 1; /* Allow the menu to grow and shrink as needed */
}
```

17. Cómo crear un menú superior

Ejemplo para crear un menú superior, utilizando una lista, cambiándole la orientación, e indicando cómo separar sus elementos.

```
<header class="navbar">
  <nav>
    <ul>
      <li><a href="#">Inicio</a></li>
      <li><a href="#">Acerca</a></li>
      <li><a href="#">Servicios</a></li>
      <li><a href="#">Contacto</a></li>
    </ul>
  </nav>
</header>
... CSS
.navbar {
```

```

padding: 1em;
}
nav ul {
  list-style: none; /*quitamos los puntos*>
  display: flex; /*indicamos como se muestra, en elcontenedor*/
  gap: 10px;
}

.navbar nav a {
  text-decoration: none; /*quita la decoración de los enlaces*/
  padding: 0.5em;
}

```

18. Cómo organizar los elementos con Grid

Supera las limitaciones de flex, porque permite alinear en dos dimensiones.

display: grid o inline-grid;

Se pueden anidar, y también hacer subgrids. Permiten flexibilidad total y acceso a todos sus elementos. Esto es un resumen, muy simple de las características más básicas.

Se define el grid explícito, con los comandos:

- grid-auto-flow: row/column /dense: rellenar huecos en blanco si entran en la cuadrícula.
- grid-template-columns: 50px tamaño / 1fr / nfr .. ; (define el número de columnas)
- grid-template-rows: 200px 75px; (define el número de filas)

Se puede usar: tamaño, fr, auto, minmax(100px,500px) .. e incluso hacerlo de manera automática con repeat (nelementos, opcion) ej. repeat (4, 1fr)

El grid implícito, son elementos que no hemos definido a mano,

- grid-auto-row: (para definir el alto para los elementos nuevos).
- grid-auto-column: para definir el ancho ..

Existe la posibilidad de definir automáticamente los elementos, y que se autoajusten sin decirlo explícitamente con auto-fit o auto-fill con repeat.

grid-template-columns: repeat (auto-fit, minmax(100px,500px))

Cada ítem: tiene una posición: Acceder a una celda: (grid-item:first-child)

- grid-column-start:1/ span 2
- grid-row-start:1 /span 3
- grid-column-end:3;
- grid-row-end:3;

Ejemplo:

```
<div class="grid"> <!-- contenedor -->
```

```

    <div class="item item-1">Item 1</div> <!-- cada uno de los ítems
del grid -->
    <div class="item item-2">Item 2</div>
    <div class="item item-3">Item 3</div>
    <div class="item item-4">Item 4</div>
</div>

```

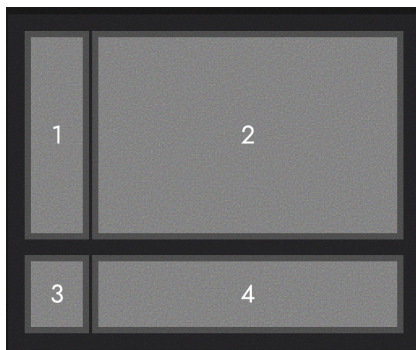
...

Ej1. Definimos 4 celdas, dando tamaño a columnas y después a filas

```

.grid {
  display: grid;
  grid-template-columns: 50px 300px;
  grid-template-rows: 200px 75px;
}

```



Definimos de forma repetitiva, 2 celdas, dando un tamaño mínimo y máximo que puede ampliarse.

```

.grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(400px, 600px));
  gap: 5px;
}

```

Definimos de forma repetitiva, n celdas, indicando tamaño mínimo y máximo. Esta es la mejor opción cuando quieres que sea responsive y permitas el movimiento de elementos.

```

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  background: grey;
  gap: 10px;
}

```

19. Como hacer un elemento flotante

La propiedad float: none/left/right/inline-start/inline-end;

Permite ubicar un elemento al lado izquierdo o derecho de su contenedor, haciendo así aparecer los elementos de texto a su costado.

20. Crear una versión para imprimir

Para crear una versión para imprimir de una página web, podemos realizar varias acciones. La más sencilla para esta práctica es crear una nueva hoja de estilos, donde indicar qué elementos de la página web original no se van a visualizar, y qué cambios en otros elementos, vamos a realizar. En general, lo que se suele hacer es sobre la misma página, indicar en el atributo "media", que en la versión de "print", se cargue una nueva hoja de estilo.

```
<link rel="stylesheet" href="estilos/estilo.css" media="screen"
/>
<link rel="stylesheet" href="estilos/estilo_imprimir.css"
media="print" />
```

(Nota: en esta práctica, te pido explícitamente que crees una versión para imprimir, pero html, por lo que deberás duplicar la página película, y sobre esta, aplicar la nueva hoja de estilos).

21. Cómo cambiar el grosor de una tipografía

El atributo `font-weight` permite cambiar el grosor de una tipografía.

Valores: normal (400) , bold (700), lighter, bolder, ... o dar el valor.

22. Ocultar elementos visualmente

`display:none;` → la visualización del elemento se desactiva, y sus hijos también. El documento se representa como si no existiera. Y desaparece del DOM, aunque puedes acceder al el. Es el mejor recurso para hacer los cambios en la versión de imprimir.

`visibility: hidden;` → permite indicar si es o no visible el elemento, pero no afecta al diseño del elemento. El espacio requerido por el modelo permanece.

`opacity: 0` → significa que los elementos, si están pero son transparentes. Se puede interaccionar con ellos.

23. Pasar una imagen a GRIS

Le aplicamos a la imagen el siguiente filtro:

```
filter: grayscale(100%);
```

24. Hacer que la pantalla no se haga más larga, sino que aparezca el scroll.

Es único por contenedor, y se usa el atributo

```
overflow: auto;
```

25. Evitar que disminuya el tamaño de un contenedor, o aumente el tamaño hasta un determinado punto

Se usan las propiedades min-width, min-height, max-width y max-height

```
min-width: 20vw;
```

26. Truco para cambiar las imágenes con CSS

La idea es simular una imagen usando un div, y ponerle a su atributo background-image, una imagen diferente, según nos interese. Por tanto, el código quedaría así:

- En el html:

```
<div class="logo" id="logo">
```

- En el CSS:

```
#logo{  
    background-image: url(../images/UGRAFILMS_bc.png);  
    background-size: 100px 100px;  
    width: 100px;  
    height: 100px;  
}
```